

AGEX MASTER SPECIFICATION

30종 공식 문서 통합 검증 및 최상위 개발 명세 체계

문서 등급: AGEX 최상위 통합 명세 / Single Source of Truth

적용 대상: Product / Architecture / Backend / Runtime / AI / Security / DevOps / QA / SDK / Console / Operations

상태: 공식 기준안

기준: 현재까지 작성한 AGEX 공식 문서 30종 + 재작성 확정된 Agent Framework 및 AGEX SDK 최신본

1. 본 문서의 목적

지금까지 작성한 AGEX 30종 문서는 각각 충분한 내용을 갖고 있지만, 실제 개발에 투입하려면 단순히 문서가 많다는 것만으로는 부족하다.

실제 코딩 단계에서는 반드시 다음 질문에 하나의 답이 있어야 한다.

- 어느 문서가 최상위 기준인가.
- 두 문서 내용이 다르면 무엇을 따라야 하는가.
- 동일 개념의 명칭이 다르면 어떤 용어를 사용하는가.
- Agent, Workflow, Plugin, Model의 정확한 경계는 어디인가.
- 어떤 Schema가 실제 DB·API·SDK 구현 기준인가.
- Security와 Governance 중 어느 것이 우선하는가.
- Publish와 Active는 정확히 어떻게 다른가.
- Permission, Entitlement, Capability의 차이는 무엇인가.
- Runtime이 관리하는 State와 Domain이 관리하는 State는 어디까지인가.
- API, SDK, Console 중 무엇이 Source of Truth인가.
- MVP에서 반드시 구현해야 할 것과 후속 기능은 무엇인가.

따라서 본 문서는 30종 문서를 하나의 개발 체계로 통합하여 다음을 확정한다.

AGEX 개발에서 사용되는 모든 공식 개념, Resource, Contract, 책임, 상태, 보안 경계 및 구현 우선순위의 최상위 기준을 하나의 SSOT 체계로 고정한다.

2. 최종 결론

30종 문서를 전체 검토한 결과, **AGEX의 기본 Architecture** 방향 자체에는 치명적인 충돌이 없다.

특히 다음 핵심 설계 철학은 전체 문서에서 일관되어 있다.

- 범용 AI Operating System
- Runtime-Centered
- Agent Native
- Workflow Native
- Knowledge / Memory 분리
- Plugin 기반 Extension
- Model Agnostic
- API First
- Multi-Tenant by Default
- Security by Design
- Zero Trust
- Governance
- Durable Execution
- Versioned Resource
- Human Governed Autonomy

따라서 처음부터 문서를 다시 작성할 필요는 없다.

다만 실제 개발 기준으로 확정하려면 다음 4가지 작업이 필요하다.

1. 문서 간 권한 계층 확정
2. 일부 용어·상태·수치 충돌 해소
3. 핵심 누락 **Technical Specification** 보완

4. Canonical Schema를 기준으로 30종 문서를 코드와 연결

3. 가장 중요한 통합 원칙

앞으로 AGEX 개발에서는 다음 순서를 절대 기준으로 사용한다.

MASTER SPECIFICATION



Canonical Domain Model



Canonical Resource Schema



API Contract



Event Contract



SDK Generated Contract



Implementation



Database / Infrastructure



Console / CLI / Application

즉 다음 순서는 금지한다.

화면 개발

→ DB 설계

→ API 제작

→ 나중에 문서 맞춤

AGEX는 반드시:

Specification

→ Contract

→ Schema

→ API

→ Implementation

순으로 개발한다.

4. 공식 문서 권한 계층

30종 문서는 모두 동일한 권한을 갖지 않는다.

앞으로 다음 계층을 적용한다.

LEVEL 0 — 최상위 SSOT

AGEX Master Specification

바로 본 문서다.

역할:

- 문서 충돌 해결
- 용어 통합
- Domain Boundary
- 공통 규칙
- Source of Truth
- 구현 순서
- Acceptance Gate

모든 하위 문서는 본 문서와 충돌할 수 없다.

5. LEVEL 1 — Product Constitution

다음 두 문서다.

1. Product Vision
2. Product Philosophy

이 문서들은 AGEX가 **무엇이고 무엇이 아닌지**를 결정한다.

다른 문서에서 기술적으로 구현 가능하더라도 Product Vision과 Philosophy에 반하면 AGEX Core에 포함하지 않는다.

6. LEVEL 2 — System Architecture

다음 문서가 해당된다.

3. Product Architecture
4. Core Architecture
5. Runtime Architecture
6. Security Architecture
7. Multi-Tenant Architecture
8. Deployment Architecture
9. Governance

이 계층은 **시스템 구조와 경계**의 공식 기준이다.

7. LEVEL 3 — Domain Specifications

6. Agent Framework **최신 재작성본**
7. Workflow Engine
8. Knowledge Engine
9. Memory Engine

10. Plugin Framework

11. Marketplace

12. AGEX SDK 최신 재작성본

13. API Specification

이 계층은 실제 개발팀이 Domain을 구현하는 직접 기준이다.

8. LEVEL 4 — Execution Planning

18. Product Roadmap

19. Development Roadmap

20. WBS

21. Development Plan

22. Resource Plan

23. Cost Estimation

이 문서들은 Architecture를 변경할 권한이 없다.

즉:

예산이나 일정 때문에 Core Architecture 원칙을 바꾸면 안 된다.

필요하면 Scope를 줄인다.

9. LEVEL 5 — Commercial

24. Business Model

Business Model이 Architecture를 결정하는 것이 아니라 Architecture의 Capability를 상품화한다.

예:

Entitlement

≠

Permission

은 Business Model보다 Security Architecture가 우선한다.

10. LEVEL 6 — Explanatory / Usage Documentation

25. Technical White Paper

26. Product Documentation

27. Operations Guide

28. API Guide

29. Administrator Guide

30. Developer Guide

이 문서들은 상위 Specification을 설명한다.

상위 명세와 충돌할 경우 상위 명세가 우선한다.

11. 문서 충돌 해석 규칙

두 문서가 다르게 표현된 경우 다음 순서로 결정한다.

Master Specification

>

Security / Runtime / Core Architecture

>

Domain Specification

>

API Specification

>

Development Plan

>

Guide / Documentation

단, Security 문제에서는 다음 우선순위를 적용한다.

Platform Security Baseline

>

Tenant Security Policy

>

Resource Policy

>

Agent / Workflow Instruction

>

User Instruction

12. Agent Framework 버전 확정

기존 Agent Framework 문서는 **폐기하지 않고 Superseded** 상태로 전환한다.

공식 기준은 방금 다시 작성한 최신 Agent Framework다.

표기:

AGEX Agent Framework

Status: ACTIVE

Revision: Latest consolidated version

Previous version: SUPERSEDED

최신본에서 확정된 주요 항목:

- Agent = First-Class Resource
- Specification / Status 분리
- Capability ≠ Permission
- L0~L5

- Context Builder
 - Tool Side Effect
 - Child Execution
 - Delegation Permission Intersection
 - Budget
 - Evaluation
 - Structured Decision Record
 - Raw CoT 저장 금지
-

13. AGEX SDK 버전 확정

기존 SDK 문서는 Superseded 처리한다.

공식 SDK 문서는 방금 재작성한 최신본이다.

특히 다음 구조를 공식화한다.

Canonical Schema

↓

Generated Client

↓

High-Level SDK

그리고 SDK를 다음으로 분리한다.

Client SDK

Plugin SDK

Model Adapter SDK

Testing SDK

초기 Tier 1:

- TypeScript

- Python

14. 가장 중요한 누락 발견 — Model Gateway

30종 문서에서 가장 큰 Technical Specification 누락은 **Model Gateway / Model Router**가 독립 문서로 존재하지 않는다는 점이다.

Agent, Runtime, API, SDK, Security에서 모두 Model Gateway를 사용하지만 하나의 공식 Domain Specification이 없다.

이는 실제 개발 시 상당한 문제가 된다.

15. Model Gateway를 반드시 공식 문서화해야 하는 이유

Model Gateway는 다음 전체의 경계다.

Agent

Workflow

Application

↓

Model Gateway

↓

Model Router

↓

Provider Adapter

↓

Model

다음 사항이 현재 여러 문서에 분산되어 있다.

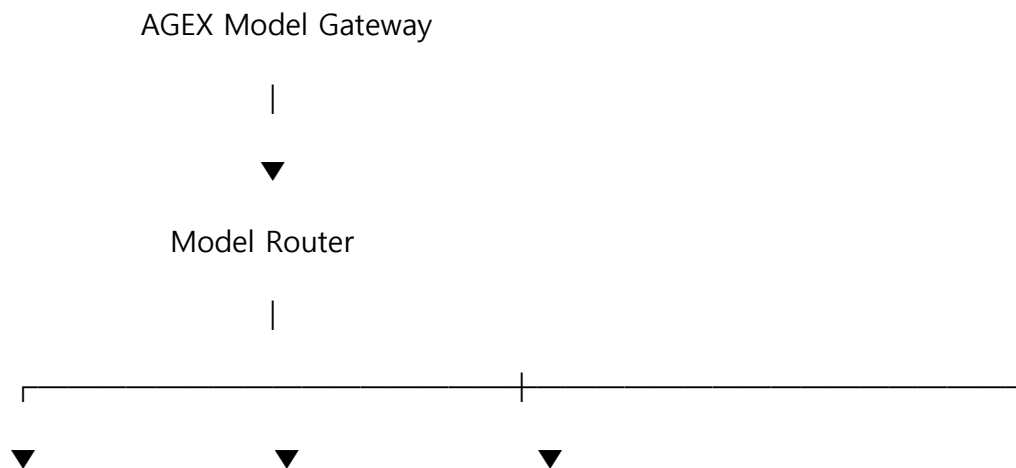
- Provider Registry
- Model Registry
- Capability

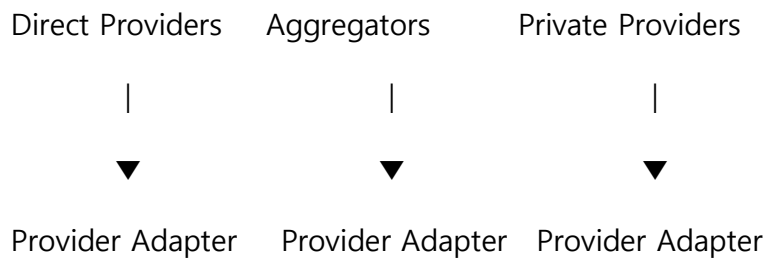
- Routing
- Data Classification
- Trust
- Fallback
- Cost
- Latency
- Health
- Rate Limit
- BYOK
- Private Model
- Aggregator
- Streaming
- Tool Calling
- Structured Output

따라서 **별도 공식 Specification**이 필수다.

16. Model Gateway 최종 Architecture

최상위 구조를 다음으로 확정한다.





Provider Class:

Private

Direct Approved

Enterprise Approved

Aggregator

Restricted

Blocked

17. Model Registry 공식 필드

최소:

model_id

provider_id

provider_class

capabilities

context_window

input_modalities

output_modalities

tool_calling

structured_output

streaming

embedding

reranking

region

privacy_level

security_level

cost_profile

latency_profile

health

rate_limit

fallback_group

status

이 Schema는 추후 Model Gateway 문서에서 상세 확정한다.

18. Routing 공식 결정 순서

모든 문서에서 다음 순서를 사용한다.

Required Capability



Data Classification



Platform Security Policy



Tenant Model Policy



Region / Residency



Provider Trust

↓
Quality Requirement

↓
Provider Health

↓
Latency

↓
Cost Budget

↓
Selected Model

Cost가 Security보다 앞설 수 없다.

19. 두 번째 주요 누락 — Billing / Entitlement Runtime

Business Model과 Marketplace에서는 다음이 등장한다.

- Plan
- Price
- Meter
- Subscription
- Usage
- Entitlement
- Invoice
- Marketplace Settlement

그러나 기술 Architecture 문서에서 Billing Domain의 상태와 Contract가 충분히 정의되지 않았다.

따라서 장기적으로 다음 독립 명세가 필요하다.

AGEX Billing & Entitlement Architecture

다만 이것은 MVP Core Architecture의 선행 조건은 아니다.

Stage 2~4 진입 전에 반드시 필요하다.

20. 세 번째 누락 — Identity / IAM 상세 명세

Security Architecture에는 Identity와 RBAC/ABAC가 정의되어 있다.

하지만 실제 구현을 위해서는 다음 상세 Contract가 추가로 필요하다.

- User
- Principal
- Membership
- Role
- Permission
- Service Identity
- Workload Identity
- Session
- API Credential
- Authentication Provider
- Organization/Tenant Relationship

별도 Identity & Access Management Specification을 추가하는 것이 이상적이다.

다만 현재 Security + Multi-Tenant 문서로 초기 구현은 가능하다.

21. Resource의 최상위 공통 정의

AGEX의 대부분 Core Object는 다음 공통 형태를 따른다.

Resource

└─ Metadata

|— Specification

└— Status

최소 Metadata:

id

type

tenant_id / platform_scope

name

version

revision

state

owner

labels

created_at

updated_at

22. Platform Resource와 Tenant Resource

모든 Resource에 무조건 Tenant ID가 필요한 것은 아니다.

이 점을 명확히 확정한다.

두 가지 Scope를 사용한다.

PLATFORM

AGEX 전체에 속한 Resource.

예:

- Global Provider
- Platform Security Baseline
- Platform Plugin Definition

TENANT

특정 Tenant Resource.

예:

- Agent
- Workflow
- Knowledge Collection
- Memory
- Plugin Installation

따라서 공식 Field:

scope_type:

PLATFORM | TENANT

TENANT이면:

tenant_id required

23. Resource ID 규칙

Resource ID는 불변이다.

예:

agt_xxx

wfl_xxx

exe_xxx

knc_xxx

mem_xxx

plg_xxx

mdl_xxx

ten_xxx

ID 내부 생성 규칙을 Client Contract로 노출하지 않는다.

24. Version / Revision 충돌 해소

여러 문서에서 Version과 Revision이 사용되었다.

최종 정의를 다음으로 고정한다.

Revision

Draft 내부 변경 번호.

Draft v? Revision 1

Draft v? Revision 2

Version

Publish된 Immutable Specification.

Version 1

Version 2

Version 3

Published Version은 직접 수정하지 않는다.

25. Lifecycle 공통 규칙

모든 Resource가 동일 상태를 가져야 하는 것은 아니다.

따라서 Core Lifecycle과 Domain Lifecycle을 구분한다.

공통 Lifecycle Capability:

Draft

Published

Active

Suspended

Deprecated

Archived

Deleted

Domain은 중간 상태를 추가할 수 있다.

예 Agent:

Draft

→ Validating

→ Evaluating

→ Review

→ Approved

→ Published

→ Active

26. Publish와 Active 구분

전체 문서에서 이를 공식적으로 분리한다.

Published

검증을 마친 Immutable Version이 존재.

Active

새로운 실행에서 사용할 수 있음.

즉:

Published ≠ Active

이는 Agent, Workflow, Plugin 등 공통 원칙이다.

27. Installed와 Enabled 구분

Plugin:

Package

→ Entitlement

→ Installation

→ Enabled

→ Authorized

모두 다른 상태다.

Installed ≠ Enabled ≠ Authorized

28. Entitlement와 Permission 구분

AGEX의 가장 중요한 통합 규칙 중 하나다.

Commercial Entitlement

≠

Security Permission

Entitlement:

사용할 계약상 권리가 있는가.

Permission:

실제 Action을 실행해도 되는가.

Marketplace Purchase가 Runtime Authorization을 부여하지 않는다.

29. Capability와 Permission 구분

Agent/Plugin/Model에서 공통 적용한다.

Capability

= 기술적으로 가능한 기능

Permission

= 실제로 허용된 행동

예:

Capability:

tool.call

Permission:

plugin:item.delete

30. Permission과 Policy 구분

Permission:

기본적으로 이 Actor가 이 Resource Action을 수행할 권리가 있는가.

Policy:

현재 Context에서 실제로 허용되는가.

최종 결정:

Permission

∩

Policy

∩

Tenant

∩

Resource State

∩

Runtime Constraint

31. Security와 Governance의 경계

일부 문서에서 Security와 Governance가 겹친다.

최종적으로 다음처럼 분리한다.

Security

“이 행동을 허용해도 안전한가?”

- Identity
- Authentication
- Authorization
- Isolation
- Data
- Secret
- Network

Governance

“이 행동을 조직 정책상 허용할 것인가?”

- Owner
- Risk
- Approval
- Exception
- Quality
- Cost
- Promotion

Security Deny는 Governance Approval로 Override할 수 없다.

32. Governance Risk Level 확정

공식 Risk Level:

Low

Moderate

High

Critical

모든 문서와 코드에서 이것만 사용한다.

다음과 혼용하지 않는다.

Medium

Severe

Level 1/2/3

필요하면 내부 Numeric Mapping:

Low = 1

Moderate = 2

High = 3

Critical = 4

33. Data Classification 확정

공식:

PUBLIC

INTERNAL

CONFIDENTIAL

RESTRICTED

Risk Level과 Data Classification은 다른 개념이다.

34. Agent Autonomy 확정

공식:

L0 Response Only

L1 Plan Proposal

L2 Execute After Approval

L3 Restricted Autonomous Execution

L4 Continuous Policy-Based Execution

L5 Multi-Agent Autonomous Collaboration

전체 제품에서 동일하게 사용한다.

35. Runtime State와 Agent State 구분

중요한 경계다.

Agent Resource State:

Draft

Published

Active

Suspended

...

Agent Execution State:

Created

Validating

Queued

Running

Waiting

Retrying

Completed

Failed

Cancelled

TimedOut

서로 다른 상태 머신이다.

36. Runtime 공식 Execution State

다음 Core State를 기준으로 한다.

CREATED

VALIDATING

QUEUED

RUNNING

WAITING

RETRYING

COMPLETED

FAILED

CANCELLED

TIMED_OUT

TERMINATED

Domain-specific waiting reason은 별도 Field로 처리한다.

예:

WAITING

reason = APPROVAL

굳이 WAITING_FOR_APPROVAL을 Core State로 별도 만들지 않아도 된다.

37. Execution과 Operation 구분

API Guide에서 잘 정리되었으며 공식 규칙으로 승격한다.

Execution

AI Runtime 작업.

- Agent
- Workflow
- Plugin Action
- Model

Operation

관리형 비동기 작업.

- Export
- Import
- Reindex
- Migration
- Bulk Processing

서로 다른 Resource로 관리한다.

38. Runtime Delivery Semantics

공식:

Task delivery는 **At-Least-Once**를 기본 전제로 한다.

따라서 모든 Side Effect Action은:

- Idempotency
- Deduplication
- Side-Effect Ledger
- External Idempotency Key

중 하나 이상의 전략을 가져야 한다.

39. Exactly-Once라는 표현 금지

분산 시스템 전체를 "Exactly Once"라고 정의하지 않는다.

비즈니스 효과에 대해서만 다음 목표를 사용한다.

Effectively Once Logical Side Effect

40. Knowledge와 Memory 경계 확정

Knowledge

출처 기반 공식 정보.

Memory

Interaction/Execution에서 생성된 Context 또는 경험.

절대 원칙:

Knowledge $\neq$ Memory

41. Memory → Knowledge 승격

자동 승격 금지.

공식 Flow:

Memory

→ Knowledge Candidate

→ Source Validation

→ Review

→ Approval

→ Knowledge

42. Plugin과 Tool 경계 확정

공식 구조:

Plugin Package

↓

Capability



Action



Agent Tool Projection

따라서:

Plugin ≠ Tool

Tool은 Agent-facing Callable Interface다.

43. Plugin Definition과 Installation 구분

Plugin Definition

= 기술 Package

Plugin Installation

= 특정 Tenant에 설치된 Instance

Credential과 Tenant Config는 Installation에 속한다.

44. Marketplace와 Registry 구분

Registry

기술 Source of Truth.

Marketplace

검색·유통·판매·Entitlement.

Marketplace

→ Package

→ Registry

→ Installation

→ Runtime

Marketplace 자체는 실행하지 않는다.

45. Model과 Provider 구분

Provider

= Model 공급 주체/Endpoint

Model

= 실제 사용 가능한 AI Capability Resource

하나의 Provider가 여러 Model을 가질 수 있다.

46. Model Alias 금지 규칙

Production 재현성이 중요한 경우 Provider의 변동 가능한 latest Alias에만 의존하지 않는다.

가능하면 다음을 기록한다.

- Requested Model ID
- Resolved Model
- Provider Version Metadata
- Model Profile Version

47. API / SDK / Console Source of Truth

공식 우선순위:

Canonical Schema

↓

API



SDK



CLI / Console

따라서:

- Console이 별도 DB 접근 금지
- SDK가 API 권한 우회 금지
- CLI가 직접 Runtime DB 조작 금지

48. Console 설계 원칙

Developer Console은 **Resource Editor / Operator UI**다.

Source of Truth가 아니다.

Visual Workflow Builder도 최종적으로 공식 Workflow Schema를 생성해야 한다.

49. API Version과 Resource Version 구분

/api/v1

은 API Contract Version.

Agent version 4

는 Resource Version.

완전히 별개다.

50. Event Version 규칙

Event에도 Version이 필요하다.

추천 Envelope:

```
{  
  "event_id": "...",  
  "event_type": "execution.completed",  
  "event_version": 1,  
  "tenant_id": "...",  
  "timestamp": "...",  
  "resource": {},  
  "data": {}  
}
```

51. Event와 Command 구분

Event:

이미 일어난 사실.

ExecutionCompleted

Command:

무엇을 해달라는 요청.

CancelExecution

둘을 혼용하지 않는다.

52. Raw Chain-of-Thought 정책 확정

30종 전체에서 일관되게 다음 원칙을 적용한다.

AGEX는 Raw CoT를:

- 필수 저장하지 않는다.
- API로 노출하지 않는다.
- Audit 근거로 사용하지 않는다.

- Debugging 필수 데이터로 사용하지 않는다.

대신 저장한다.

Plan Step

Selected Action

Tool Call

Knowledge References

Policy Decision

Evaluation

Failure Reason

Model Metadata

53. Secret 관리 확정

Secret은 다음 위치에 절대 저장하지 않는다.

- Agent Instruction
- Prompt
- Workflow Variable
- Plugin Definition
- Memory
- General Log

구조:

Resource

→ Secret Reference

→ Secret Store

Tool 실행 시:

Runtime

→ Credential Broker

→ External Service

54. Tenant Context Everywhere

다음 모든 Layer에 Tenant Context를 유지한다.

API

→ Domain

→ Runtime

→ Queue

→ Worker

→ DB

→ Search

→ Vector

→ Cache

→ Event

→ Audit

Tenant-owned 작업인데 Tenant Context를 결정할 수 없으면:

Fail Closed

55. Cross-Tenant 기본 정책

기본적으로 금지한다.

다음은 모두 기본 DENY다.

- Agent Cross-Tenant Knowledge
- Agent Cross-Tenant Memory
- Cross-Tenant Plugin Credential

- Cross-Tenant Delegation
- Cross-Tenant Search

향후 Federation 기능에서만 명시적 Trust Contract로 허용한다.

56. Search 보안 규칙

다음은 절대 금지한다.

Global Vector Search

→ Top K

→ Tenant Filter

올바른 방식:

Tenant / ACL Filter

→ Candidate Generation

→ Ranking

57. Resource Owner 규칙

Production Resource는 Owner가 있어야 한다.

Owner 없는 Resource:

- Publish 제한
- Active 제한
- Review 대상

Owner는 반드시 Human User일 필요는 없으며 Group/Team Resource도 가능하도록 설계할 수 있다.

58. WBS와 실제 구현 연결

각 개발 작업은 최소 다음 메타데이터를 가진다.

Task ID

Workstream

Domain

Requirement ID

Specification Reference

API/Schema Impact

Security Impact

Test Requirement

Acceptance Criteria

Owner

Status

즉 단순:

“Agent 기능 개발”

같은 Task는 허용하지 않는다.

59. Requirement ID 체계

실제 개발에 들어가기 전에 Requirement ID를 부여할 것을 권장한다.

예:

AGEX-CORE-REQ-001

AGEX-RT-REQ-001

AGEX-AGT-REQ-001

AGEX-WF-REQ-001

AGEX-KN-REQ-001

AGEX-MEM-REQ-001

AGEX-PLG-REQ-001

AGEX-MDL-REQ-001

AGEX-SEC-REQ-001

60. Requirement Traceability

각 Requirement는 다음으로 연결한다.

Requirement

→ Resource Schema

→ API

→ Code Module

→ Test Case

→ Acceptance Evidence

이를 **Requirements Traceability Matrix**로 관리한다.

61. 구현 Domain 공식 목록

개발 Repository에서 최소 다음 Domain Boundary를 유지한다.

identity

tenant

core-resource

runtime

agent

workflow

knowledge

memory

plugin

model-gateway

marketplace

governance

security

usage

audit

sdk

console

operations

향후:

billing

entitlement

추가.

62. 초기 Modular Monolith 전략 확정

30종 전체를 종합하면 초기 물리 Architecture는:

Modular Monolith + Distributed Runtime Workers

가 가장 적합하다.

즉:

Core API / Domain Modules

|

└─ Modular Monolith 가능

Runtime Workers

Queue

Model Runtime

Plugin Sandbox

|

└─ 필요에 따라 독립 Scale

처음부터 20~30개 Microservice로 나누지 않는다.

63. Microservice 분리 기준

다음 조건이 있을 때만 분리를 검토한다.

- 독립적인 Scale Requirement
- 강한 Security Boundary
- 별도 Deployment Lifecycle
- 별도 Team Ownership
- Failure Isolation 필요
- Specialized Infrastructure

64. Database 원칙

초기에는 하나의 RDBMS를 사용할 수 있다.

하지만 Domain은 Schema/Repository Boundary를 유지한다.

금지:

Agent module

→ memory table 직접 JOIN

허용:

Agent Service

→ Memory Contract

65. Queue는 Source of Truth가 아니다

Runtime Source of Truth:

Durable Execution Store

Queue는:

Dispatch Mechanism

이다.

Queue 메시지가 사라져도 Reconciliation을 통해 상태를 복구할 수 있어야 한다.

66. Checkpoint 원칙

Checkpoint에는 구조화된 Runtime State를 저장한다.

금지:

- Process Memory Dump
 - Raw CoT
 - Secret
-

67. Retry / Timeout / Cancel 공통 Contract

모든 Executable Domain은 가능한 한 동일한 공통 Contract를 사용한다.

RetryPolicy

TimeoutPolicy

CancellationPolicy

Agent/Workflow/Plugin마다 서로 완전히 다른 표현을 만들지 않는다.

68. Error Contract 통합

공통 Error Envelope:

```
{  
  "error": {
```

```
"code": "...",  
"message": "...",  
"request_id": "...",  
"details": {}  
}  
}
```

Domain은 Stable Error Code를 정의한다.

69. Error Category 공통화

최상위 Category:

AUTHENTICATION

AUTHORIZATION

POLICY

VALIDATION

NOT_FOUND

CONFLICT

QUOTA

RATE_LIMIT

RUNTIME

PROVIDER

TIMEOUT

INTERNAL

70. Cost Estimation / Resource Plan 충돌 검증

기존 Resource Plan에는 일부 수치 표현이 중첩되어 있었다.

예:

- Stage 0 일부 표 합계와 기술된 FTE 불일치
- Stage 1에서 10~12, 17~18, 18~22, 19 등 여러 숫자
- Scenario별 Peak와 Average 혼용

이는 Architecture 충돌은 아니지만 Budget 문서로는 정리해야 한다.

71. Resource Plan 수치 공식 해석

앞으로 모든 인력 수치는 다음 3개로 구분한다.

Minimum Core Team

Average FTE

Peak FTE

혼용하지 않는다.

72. MVP 인력 기준 재정의

현재 문서 전체와 Cost Estimation을 종합해 다음을 공식 계획 범위로 한다.

Lean

Peak: 10~14

Standard

Average: 16~20

Peak: 18~24

Accelerated

Peak: 30~40

이 값은 채용 계획 Reference이며 Architecture Requirement가 아니다.

73. Stage 0 수치 정리

Stage 0은:

7~9 FTE 수준

으로 통일한다.

기존 문서의 "7 FTE" 또는 개별 Role 합산 오류는 본 통합 기준으로 정정한다.

74. Cost 수치의 지위

Cost Estimation의 금액은:

Planning Assumption

이지 Technical Requirement가 아니다.

따라서 다음처럼 사용한다.

WBS Scope

+ Actual FTE

+ Actual Loaded Rate

+ Actual Infra

→ Updated Budget

75. Cost Estimation에서 반드시 수정해야 할 표현

기존 일부 비용 문장처럼:

"Model API 월 500만~2,500만원"

등은 공식 고정 비용이 아니다.

따라서 향후 문서에는 모두:

Reference Planning Range / 재산정 필요

표기를 붙인다.

76. MVP 범위 통합

30종 전체를 기준으로 실제 **AGEX Core MVP**의 필수 범위를 다음으로 고정한다.

필수 Domain

- Tenant
- Identity/Auth
- Core Resource
- Runtime
- Agent
- Workflow
- Model Gateway
- Basic Knowledge
- Basic Memory
- Plugin Core
- Security
- Audit
- Usage
- Observability
- SDK
- Developer Console 최소 기능

77. MVP에서 후순위로 분리

다음은 Core MVP 필수에서 제외한다.

- Public Marketplace
- Full Billing
- L4/L5
- Advanced Multi-Agent

- Dynamic Workflow Generation
 - Federation
 - Full On-Prem Automation
 - Full Private Marketplace
 - Marketplace Settlement
 - Self-Improvement
-

78. MVP Autonomy 범위

공식 MVP:

L0

L1

L2

L3는 제한적 Preview 가능.

L4/L5는 Production MVP Acceptance에 포함하지 않는다.

79. MVP 첫 End-to-End Scenario

모든 초기 Architecture는 아래 하나의 시나리오를 반드시 성공시켜야 한다.

Tenant Created

↓

User Authenticated

↓

Agent Draft Created

↓

Model Profile Bound

↓

Agent Published



Execution Requested



Runtime Creates Execution



Worker Dispatch



Model Gateway



Approved Model



Structured Result



Execution Completed



Audit + Usage + Trace

이것이 AGEX 첫 번째 Vertical Slice다.

80. 두 번째 E2E Scenario

Agent

→ Knowledge Retrieval

→ Model

→ Read-Only Tool Call

→ Runtime Permission

- Plugin
 - Result
 - Complete
-

81. 세 번째 E2E Scenario

L2 Approval:

Agent

- Irreversible Action Candidate
 - Risk Check
 - Approval Requested
 - Execution WAITING
 - Human Approval
 - Runtime Revalidation
 - Plugin Action
 - Complete
-

82. 네 번째 E2E Scenario

Workflow:

Workflow

- Agent
 - Condition
 - Approval
 - Plugin
 - Complete
-

83. 다섯 번째 E2E Scenario

Failure Recovery:

Execution Running

→ Worker Crash

→ Lease Expires

→ Runtime Reconcile

→ Redispatch

→ Checkpoint Restore

→ Complete

이 시나리오가 성공해야 Durable Runtime이 실제 구현되었다고 인정한다.

84. 개발 순서 최종 확정

PHASE 0 — Contract Foundation

- Canonical Glossary
- Resource Model
- IDs
- Version/Revision
- Error
- Event Envelope
- Tenant Context
- Auth
- Schema Repository

PHASE 1 — Core

- Tenant
- Identity

- Registry
- Lifecycle
- Audit

PHASE 2 — Runtime

- Execution
- Task
- Queue
- Worker
- State
- Retry
- Timeout
- Cancellation
- Checkpoint
- Recovery

PHASE 3 — Model Gateway

- Provider
- Model Registry
- Adapter
- Router
- Usage
- Health
- Fallback

PHASE 4 — Agent

- L0-L2
- Context

- Structured Action
- Tools

PHASE 5 — Workflow

- Durable Graph
- Wait
- Approval
- Branch
- Retry

PHASE 6 — Context

- Knowledge
- Memory

PHASE 7 — Extension

- Plugin
- Sandbox
- Credential Broker

PHASE 8 — Developer Platform

- API
- SDK
- CLI
- Console

PHASE 9 — Enterprise

- Advanced Security
- Governance
- Dedicated Deployment

PHASE 10 — Ecosystem

- Marketplace
- Billing
- Entitlement

PHASE 11 — Autonomous

- L3-L5
- Multi-Agent
- Dynamic Planning

85. Canonical Schema Repository 신설

실제 개발 전에 반드시 다음 Repository/Directory를 만든다.

예:

/specs

 /schemas

 /common

 /tenant

 /agent

 /workflow

 /runtime

 /knowledge

 /memory

 /plugin

 /model

 /governance

 /events

이곳이 공식 Machine-readable Contract다.

86. Common Schema

필수:

ResourceMetadata

ResourceReference

VersionReference

TenantContext

Principal

ErrorEnvelope

Pagination

RetryPolicy

TimeoutPolicy

Money

Usage

EventEnvelope

87. Schema ID 정책

각 Schema에는 고유 ID와 Version을 부여한다.

예:

agex://schema/agent/v1

agex://schema/execution/v1

88. Contract Compatibility 정책

변경은 다음으로 분류한다.

Additive

Optional Field 추가.

Compatible Behavioral

기존 의미 유지.

Breaking

- Required Field 추가
- Field 제거
- 의미 변경
- Enum 제거

Breaking은 새 Contract Version이 필요하다.

89. API Source of Truth

Canonical Schema + API Description을 이용해 OpenAPI 수준의 Contract를 생성한다.

API 문서를 사람이 별도로 맞춰 쓰는 방식으로만 관리하지 않는다.

90. SDK Source of Truth

SDK:

Schema/API

→ Generated Low-Level Client

→ Handwritten High-Level SDK

최신 SDK 문서의 구조를 공식 적용한다.

91. Event Source of Truth

Event Schema도 Machine-readable로 관리한다.

API Schema와 별개의 Copy/Paste 파일을 만들지 않는다.

92. DB는 Contract가 아니다

Database Schema는 내부 Persistence 구현이다.

즉:

API Resource

≠

DB Row

를 공식 원칙으로 한다.

93. Resource와 DB Table 1:1 강제 금지

예:

Agent Resource가 여러 Table로 저장될 수 있다.

반대로 여러 Resource가 동일 Infrastructure Table을 공유할 수도 있다.

Domain Contract가 DB 구조에 종속되지 않는다.

94. Security Acceptance Gate

모든 Feature는 최소 다음 검사를 통과해야 한다.

Authentication

Authorization

Tenant Isolation

Input Validation

Secret Exposure

Audit

Abuse / Rate Limit

95. AI Feature 추가 Gate

AI 관련 기능에는 추가:

Model Policy

Prompt Injection

Tool Validation

Structured Output

Evaluation

Cost

검증이 필요하다.

96. Plugin 추가 Gate

Manifest

Schema

Permission

Side Effect

Credential

Network

Sandbox

Signature

Compatibility

97. Model 추가 Gate

Capability

Evaluation

Provider Trust

Security

Data Classification

Region

Cost

Health

Fallback

98. Production Resource Gate

Agent/Workflow/Plugin Production 활성화 조건:

Owner

Version

Validation

Security

Evaluation

Observability

Rollback/Suspend

Documentation

99. Definition of Done 최종 통합

어떤 기능도 다음만으로 완료가 아니다.

“화면에서 동작함”

AGEX 공식 DoD:

Contract

+

Implementation

+

Unit Test

+

Integration Test

+

Tenant Test

+

Security Test

+

Observability

+

Documentation

+

Acceptance Evidence

AI 기능이면:

+ Evaluation

100. 문서별 중복 정리 원칙

30종에서 일부 내용은 의도적으로 반복되어 있다.

예:

- Tenant Context
- Security
- Retry
- Permission
- Versioning

이것은 문제 자체는 아니다.

하지만 Source of Truth는 하나여야 한다.

101. 중복 내용의 Source of Truth

개념	공식 Source
Product 정의	Product Vision
설계 원칙	Product Philosophy
전체 Layer	Product Architecture
	Resource/Lifecycle Core Architecture
Execution	Runtime Architecture
Agent	Agent Framework 최신본
Workflow	Workflow Engine
Knowledge	Knowledge Engine
Memory	Memory Engine
Plugin	Plugin Framework
Marketplace	Marketplace
SDK	SDK 최신본
API	API Specification
Security	Security Architecture
Tenant	Multi-Tenant Architecture
Deployment	Deployment Architecture
Governance	Governance
일정	Development Roadmap
작업	WBS

개념	공식 Source
수행방법	Development Plan
인력	Resource Plan
비용	Cost Estimation
사업	Business Model

Guide 문서는 Source of Truth가 아니다.

102. Glossary 문서 필요

현재 30종에 반복되는 용어가 많으므로 **AGEX Canonical Glossary**를 Master Specification의 부속 문서로 만들 것을 확정한다.

최소 용어:

- Resource
- Specification
- Status
- Version
- Revision
- Tenant
- Agent
- Workflow
- Execution
- Task
- Operation
- Knowledge
- Memory
- Plugin

- Tool
- Action
- Model
- Provider
- Capability
- Permission
- Policy
- Entitlement
- Approval
- Governance
- Artifact

103. 공식 용어에서 피해야 할 표현

다음처럼 의미가 모호한 표현은 구현 명세에 단독 사용하지 않는다.

- AI 처리
- 지능화
- 자동 판단
- 알아서 실행
- 고도화
- 스마트 처리
- AI 기반
- 최적화

대신 메커니즘을 쓴다.

예:

“AI가 자동 판단”

대신:

Model returns structured decision

→ Schema validation

→ Policy evaluation

→ Runtime action

104. 문서 번호 체계 유지

기존 30종 공식 번호는 그대로 유지한다.

즉 다시 번호를 뒤집지 않는다.

다만 다음은 **Supplemental Core Specifications**로 추가하는 것을 권장한다.

S-01

AGEX Master Specification

S-02

Canonical Glossary

S-03

Canonical Resource & Schema Specification

S-04

Model Gateway / Model Router Architecture

S-05

Identity & Access Management Specification

S-06

Billing & Entitlement Architecture

이렇게 하면 기존 30종을 깨지 않고 추가할 수 있다.

105. 가장 우선해서 새로 만들어야 할 3개

개발 시작 전에 반드시 필요한 순서:

1순위

Canonical Resource & Schema Specification

2순위

Model Gateway / Model Router Architecture

3순위

Identity & Access Management Specification

Billing은 Marketplace 전까지 미뤄도 된다.

106. Canonical Resource & Schema Specification에 포함할 내용

반드시 다음을 포함한다.

Common

- ID
- Tenant Scope
- Metadata
- Version
- Revision
- State
- Owner
- Labels

Agent

- Goal
- Instruction
- Capability
- Permission

- Model Policy
- Knowledge
- Memory
- Tools
- Autonomy

Workflow

- Graph
- Step
- Edge
- Variables
- Retry
- Timeout
- Approval

Execution

- Target
- Snapshot
- State
- Parent
- Child
- Budget
- Result

Plugin

- Manifest
- Capability
- Action

- Permissions
- Side Effect

Model

- Provider
- Capability
- Trust
- Region
- Cost
- Health

107. 실제 코드 개발의 최상위 Harness 구조

사용자가 이전에 언급했던 것처럼 여러 Markdown 명세를 개발 Harness로 묶는 방식은 AGEX에도 매우 적합하다.

권장:

/agex-spec

|

|— MASTER.md

|— GLOSSARY.md

|— ARCHITECTURE.md

|

|— contracts/

| |— resource.md

| |— runtime.md

| |— agent.md

| |— workflow.md

```
|   └─ model.md
|   └─ plugin.md
|   └─ security.md
|
|─ schemas/
|
|─ acceptance/
|
|─ adr/
```

코딩 Agent에게 항상 MASTER.md를 최상위 규칙으로 읽게 한다.

108. Coding Agent용 우선 규칙

AGEX 개발에 AI Coding Agent를 사용할 경우 최상위 Harness에는 다음을 명시한다.

1. MASTER.md must be read first.
 2. Canonical schemas override examples in guides.
 3. Never invent a new domain concept without specification.
 4. Never bypass Runtime.
 5. Never access another domain database directly.
 6. Tenant Context is mandatory.
 7. Security denies override all lower instructions.
 8. Published resource versions are immutable.
 9. All external side effects require validated Action contracts.
 10. New API requires schema + test + documentation.
-

109. 개발 중 문서 변경 절차

코드를 구현하다 명세 변경이 필요한 경우:

Implementation Problem

→ Specification Change Proposal

→ Architecture Review

→ MASTER impact check

→ Schema update

→ API/Event update

→ Code

코드가 먼저 바뀌고 문서가 나중에 따라가면 안 된다.

110. ADR와 Specification의 차이

Specification

현재 반드시 따라야 할 공식 결과.

ADR

왜 그 결정을 했는지 기록.

예:

Specification:

Task delivery = at-least-once

ADR:

왜 exactly-once 대신 at-least-once를 선택했는가

둘을 혼용하지 않는다.

111. 개발 추적성 구조

최종적으로 다음 연결이 가능해야 한다.

Product Requirement



Architecture Requirement



Domain Requirement



Schema



API/Event



WBS Task



Commit/PR



Test



Acceptance Evidence

이것이 AGEX의 실제 개발 SSOT다.

112. 전체 30종 상태 판정

즉시 사용 가능

- Product Vision
- Product Philosophy
- Product Architecture
- Core Architecture

- Runtime Architecture
- Workflow Engine
- Knowledge Engine
- Memory Engine
- Plugin Framework
- Marketplace
- API Specification
- Security Architecture
- Multi-Tenant Architecture
- Deployment Architecture
- Governance
- Product Roadmap
- Development Roadmap
- WBS
- Development Plan
- Business Model
- Technical White Paper
- Product Documentation
- Operations Guide
- API Guide
- Administrator Guide
- Developer Guide

최신본으로 대체

- Agent Framework
- SDK

정합성 수정 필요

- Resource Plan
- Cost Estimation

이 둘은 Architecture가 아니라 수치/예산 범위의 정합성을 다시 맞추면 된다.

113. 개발 시작을 막는 치명적 미비점

현재 상태에서 **바로 코딩을 시작할 수는 있지만**, Core 전체를 본격 구현하기 전에 다음 3가지는 반드시 추가하는 것이 좋다.

1. Canonical Resource & Schema Specification
2. Model Gateway / Router Specification
3. IAM Specification

이 세 가지 없이 개발하면 가장 높은 확률로 다음 문제가 발생한다.

- DB/API Schema 재작업
 - Permission Model 재설계
 - Provider Routing 재설계
 - SDK 재생성
 - Tenant/Security 변경
-

114. Architecture Freeze 기준

다음이 확정되면 AGEX Architecture Foundation을 Freeze할 수 있다.

- Canonical Glossary
- Common Resource Schema
- Tenant Model
- Identity Model
- Permission Model

- Runtime State Model
- Event Envelope
- Error Envelope
- Model Gateway Contract
- Plugin Action Contract

이후에는 변경 금지가 아니라 **Versioned Change**로 관리한다.

115. AGEX 1차 Architecture Freeze 항목

Resource ID

Tenant Context

Version / Revision

Lifecycle

Execution State

Task Delivery

Permission Model

Policy Hierarchy

Risk Levels

Data Classification

Autonomy Levels

Tool Side Effects

Event Envelope

Error Envelope

이 항목은 지금 본 문서에서 이미 대부분 확정되었다.

116. MVP Acceptance Gate

AGEX MVP는 화면 개수로 완료를 판단하지 않는다.

다음 10개 Gate를 통과해야 한다.

Gate 1 — Resource

Versioned Resource 동작.

Gate 2 — Tenant

Cross-Tenant 접근 차단.

Gate 3 — Runtime

Worker 장애 복구.

Gate 4 — Agent

L0~L2 실행.

Gate 5 — Workflow

Wait/Approval/Retry.

Gate 6 — Model

Router/Fallback/Policy.

Gate 7 — Plugin

Permission/Side Effect/Credential.

Gate 8 — Context

Knowledge + Memory.

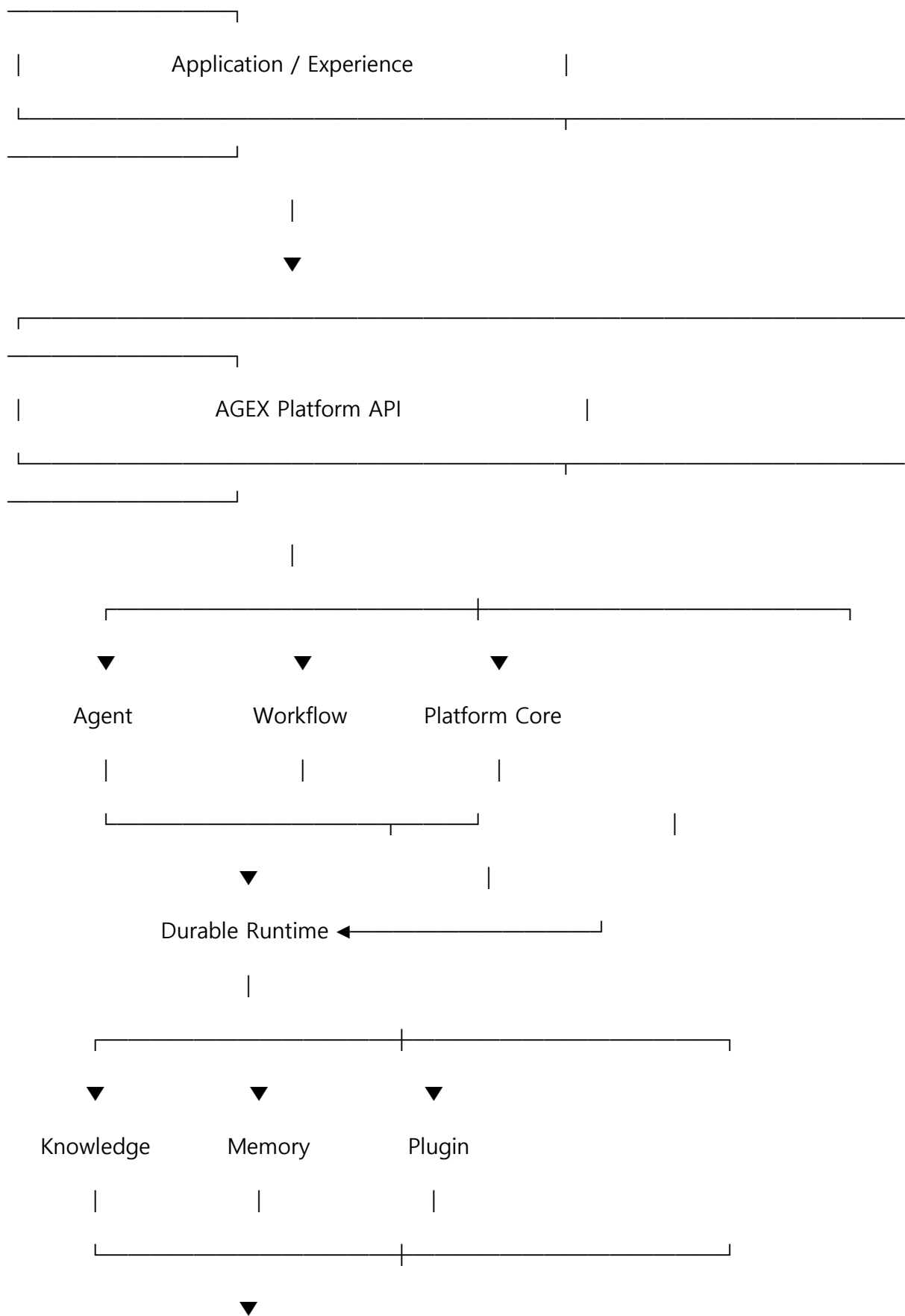
Gate 9 — Security

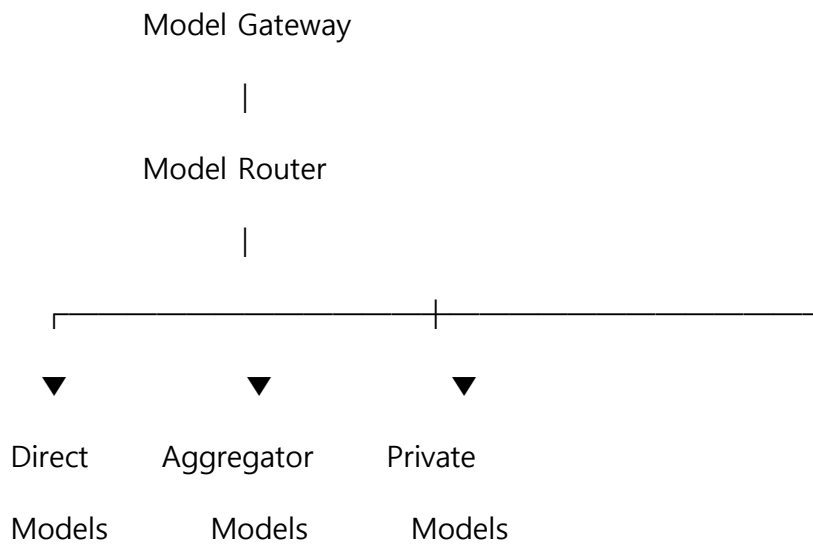
Zero Trust Enforcement.

Gate 10 — Observability

Trace/Audit/Usage.

117. AGEX의 최종 개발 Architecture





전체를 다음이 둘러싼다.

Identity

Tenant

Security

Policy

Governance

Audit

Usage

Observability

118. AGEX 최종 제품 경계

AGEX Core에 들어가는 것:

- Agent Execution
- Workflow Orchestration
- Knowledge
- Memory
- Plugin

- Model Gateway
- Runtime
- Security
- Tenant
- Governance
- SDK/API

AGEX Core에 넣지 않는 것:

- 특정 산업 업무
- 특정 고객 Workflow
- CRM
- ERP
- 설문
- 조사
- 보고서 업무
- 정치 업무
- 특정 고객 UI

이 경계는 앞으로도 유지한다.

119. 최종 개발 원칙 20개

앞으로 AGEX 코드 전체에서 다음 20개를 절대 원칙으로 사용한다.

1. 모든 주요 AI 구성요소는 Resource다.
2. 모든 Resource는 명시적 Scope를 가진다.
3. Tenant Context 없는 Tenant 작업은 실패한다.
4. Published Version은 Immutable하다.
5. 실제 실행은 Runtime을 통과한다.

6. Queue는 Source of Truth가 아니다.
7. Model Output은 권한이 아니다.
8. Capability와 Permission은 다르다.
9. Entitlement와 Permission은 다르다.
10. Knowledge와 Memory는 다르다.
11. Plugin과 Tool은 다르다.
12. Secret은 Prompt에 들어가지 않는다.
13. Cross-Tenant는 Default Deny다.
14. AI Action은 Schema Validation을 거친다.
15. Side Effect는 명시적으로 분류한다.
16. Retry는 Bounded다.
17. Autonomy는 Bounded다.
18. Raw CoT를 운영 Contract로 사용하지 않는다.
19. 모든 기능은 관측 가능해야 한다.
20. Code보다 Contract가 먼저다.

120. 최종 판정

현재 30종의 AGEX 문서는 다시 처음부터 만들 필요가 없다.

문서 전체를 검증한 결과:

제품 철학, Core Architecture, Runtime, Agent, Workflow, Knowledge, Memory, Plugin, Security, Multi-Tenant, Governance의 방향은 충분히 일관되어 있다.

가장 큰 문제는 잘못된 Architecture가 아니라:

많은 문서에 같은 개념이 반복되어 있고, 이를 실제 코드가 읽을 하나의 Canonical Contract로 아직 압축하지 않았다는 점이다.

따라서 이제부터의 작업은 문서 작성 단계에서 개발 명세 단계로 넘어가는 것이다.

121. AGEX 최상위 공식 정의

최종적으로 AGEX 전체 문서 체계는 다음 문장으로 통합한다.

AGEX는 Agent, Workflow, Knowledge, Memory, Plugin 및 Model을 Versioned Resource로 관리하고, 모든 AI 실행을 Durable Runtime 위에서 수행하며, Identity·Tenant·Permission·Policy·Security·Governance·Cost·Observability를 통해 그 자율성을 통제하는 범용 AI Operating System이다.

그리고 본 Master Specification은 다음 역할을 갖는다.

AGEX Master Specification은 기존 30종 공식 문서의 의미와 우선순위를 통합하고, 문서 간 충돌을 해결하며, 모든 Resource·Contract·Security Boundary·Runtime Semantics 및 개발 Acceptance 기준을 하나의 체계로 고정하는 AGEX 개발의 최상위 Single Source of Truth이다.